

EEE3095/6S: Practical 4

STM32 DACs

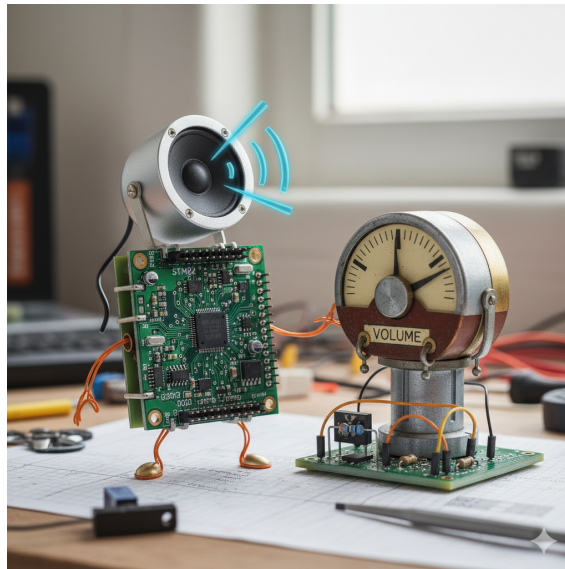
September 22, 2025

1 Overview

Finally, no more turning LEDs on and off and onto more serious stuff with real world implications.

This practical introduction introduces you to DACs. A DAC converts a digital code into an analog voltage or current and works on the general premise that $V_{OUT} = k \times DigitalInput$, where k is a proportionality factor and the **digital input** is a number in the range of $0 \rightarrow (2^{\text{bits}} - 1)$. DACs have a variety of uses, with the most important among them converting digital signals to analog audio in just about every single electronic audio system in the world. The STM Board already has a built-in DAC. Setting it up is trivial and there are plenty of online resources available if you wish to test it out.

In this practical we will be making use of a quick and dirty DAC, an amplifier circuit, look-up tables (LUTs) for a few different waveforms, set up two timers for generating the PWM signal and cycling through the LUT values generated from 3 wav files (Sample rate of 44.1 kHz) and 3 waveforms, and then use the small speaker to produce audible sound. Volume control will be implemented using an analog potentiometer and audio signals will be generated from lookup tables (LUTs) derived from .wav files and the waveforms.



2 Outcomes and Knowledge Areas

In this practical, you will be using C code (and the HAL libraries) to interface your microcontroller board with an amplifier circuit that you will build in the lab. You will also use look-up tables (LUTs) to represent sound waveforms, set up two timers on the dev board (one timer for generating the PWM signal and one for cycling through the LUT values periodically), and then feed the output to your amplifier. The aim of this is to use the LUT values to vary the CCR (Capture/Compare Register) value, effectively changing your duty cycle and generating sound. You will learn about the following aspects:

- DACs

- Simple Amplifier design
- PWM
- Pushbutton interrupts

3 Hardware

You will require the following external hardware (in addition to your STM dev board) to properly complete this practical:

- Breadboard
- Signal generator
- Components to build amplifier circuit such as resistors, capacitors, transistors, op-amps etc
- Speaker
- Potentiometer

4 Deliverables

For this practical, you must:

- Develop the code required to meet all objectives specified in the Tasks section
- Push your completed code to a shared repository on GitHub
- Demonstrate your working implementation to a tutor in the lab. Before calling a tutor for the demo submit a pdf file of the main.c code to Gradescope and for this prac every student should submit i.e it's not a group submission. You will be allowed to conduct your demo during any lab session before the practical submission deadline. **All group members will be required to be present at the demo as each group member has to answer two questions that count for 5 marks.** Name your pdf file as follows:

[EEE3096S 2025 Practical 4 Demo STDNUM001.pdf](#)

- Write a short **2-page** report documenting your **main.c** code, GitHub repo link, and a brief description of the implementation of your solutions. This must be in PDF format and submitted on Gradescope with the naming convention(**If you do not adhere to the naming convention, there will be a penalty**):

[EEE3096S 2025 Practical 4 Report STDNUM001 STDNUM002.pdf](#)

Check the Appendix for Report Structure 7.

- Your practical mark will be based both on your demo to the tutor (i.e., completing the below tasks correctly) as well as your short report.

5 Getting Started

The procedure is as follows:

1. Clone or download the Git repository(The practical folder(s) of interest is [Practical4](#)
git clone <https://github.com/EEE3096S-UCT/EEE3096S-2025.git>
2. • Open **STM32CubeIDE**, then navigate through the menus:

File → Import → Existing Code as Makefile Project

- Click Next.
- In the dialog, click Browse... and select your project folder.
- Under **Toolchain**, choose MCU ARM GCC.

- Click Finish.

6 Tasks

Complete the following tasks using the main.c file in STM32CubeIDE with the HAL libraries, and then demonstrate the working execution of each task to a tutor:

Sample Solution can be found at: [Sample](#)

Task 1

Your first task will be to:

1. Generate lookup tables (LUTs) for a single cycle of a sinusoid, a sawtooth wave and a triangular wave. Your lookup table should have a minimum of 128 values ranging from 0 to 4095 (12 bit resolution). Plot your LUTs using MATLAB/Excel/Python to ensure you have the correct wave shape. Copy your LUTs into your main.c file.
2. Generate a Lookup Table (LUT) for three sound waveforms sampled at 44.1kHz. The LUT should have at least 128 values that range from 0 to 4095 (12-bit resolution). Use MATLAB or Python to process a .wav audio file, plot the waveform to confirm accuracy, and then copy the LUT values into your main.c file. The three .wav files can be found here:

[wav_files](#)

Task 2

Assign values to NS, TIM2CLK and Fsignal. NS is the number of samples in your LUT, TIM2CLK should be set to 16 MHz (which you can see in the .ioc file), and Fsignal is the frequency that we want our analog signal to have. Find a suitable limit to the Fsignal. You will be required to explain your reasoning.

Task 3

Calculate TIM2_Ticks. This is the number of cycles that the PWM duty cycle will be changed and depends on the clock frequency TIM2CLK, number of samples NS and output frequency F signal. Note: We will be using Direct Memory Access (DMA) to change the PWM duty cycle. DMA is used to provide high-speed data transfer between peripherals and memory as well as memory to memory. Data can be quickly moved by DMA without any CPU actions. Check the .ioc file to see how the DMA is configured (as well as the other peripherals!).

Task 4

Configure the DAC to output values from the LUT periodically. Use a timer (e.g., TIM2) to cycle through the LUT and generate the waveform output. Verify that the DAC is correctly configured to produce sound through the amplifier circuit. To do this complete the following:

In the main() function:

1. Start TIM3 in PWM mode on channel 3
2. Start TIM2 in Output Compare (OC) mode on channel 1.
3. Start the DMA in interrupt (IT) mode. The source address is one of the LUTs you created earlier. The destination address is the CCR3 register for TIM3_CH3. Start with the Sine wave LUT.
4. Write the waveform type to the LCD screen, i.e., "Sine".
5. Make use of `__HAL_TIM_ENABLE_DMA(htim2, TIM_DMA_CC1)` to start the
6. DMA transfer.

HINT: The 3 functions you need for TASK 4 begin with either HAL_DMA or HAL_TIM. Press Ctrl + Space to see the options once you have typed out the first few letters.

Task 5

An external interrupt has been configured on the PA0 pushbutton(labelled Button0 in the code), and EXTI0_1_IRQHandler(void) is called when it is pressed. Write code that changes from one waveform/sound to the next when the button is pressed. Remember to debounce your button presses; debouncing should eliminate noise from bouncing, but not make the response seem sluggish if the button is pressed multiple times shortly after each other. Use `__HAL_TIM_DISABLE_DMA(htim2, TIM_DMA_CC1)` and `HAL_DMA_Abort_IT` to stop the DMA transfer before changing the source address, then re-enable DMA.

As part of your interrupt, write the current waveform type/ Song to the LCD screen, e.g., "Sine", "Sawtooth", or "Triangular" or "Piano" or "Guitar" or "Drum" depending on which LUT is selected after pressing the pushbutton.

Task 6

Build the **amplifier** circuit on your breadboard using components provided and others that you can find at White Lab if necessary. Include a **brief motivation** for your designed circuit explaining your **design choices**. Pages required to document your **circuit design** and **motivation** to be added as an **appendix** and will not count towards the total page length of the report.

Some helpful links for designing the amplifier:

1. [Resource1](#)
2. [Resource2](#)

You might want to build a simple filter to filter the output before passing it to the speaker, to better improve on the quality of the output(Though wont make much of a difference unless your filter is well designed). Warning be careful as you might end up filtering the sound itself.

NOTE: The quality of your sound largely depends on how good your amplifier is and the choice of F_SIGNAL.

The sample circuits are shown in the Figures 1, 2, 3, 4 below:

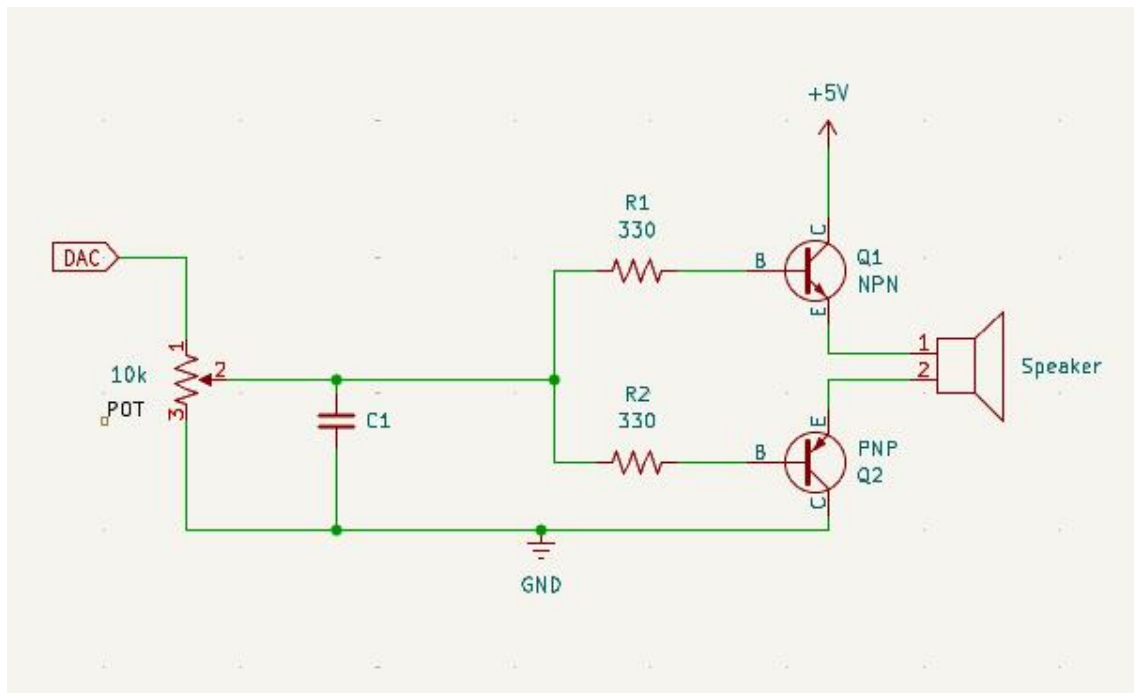


Figure 1: Amplifier Design 1

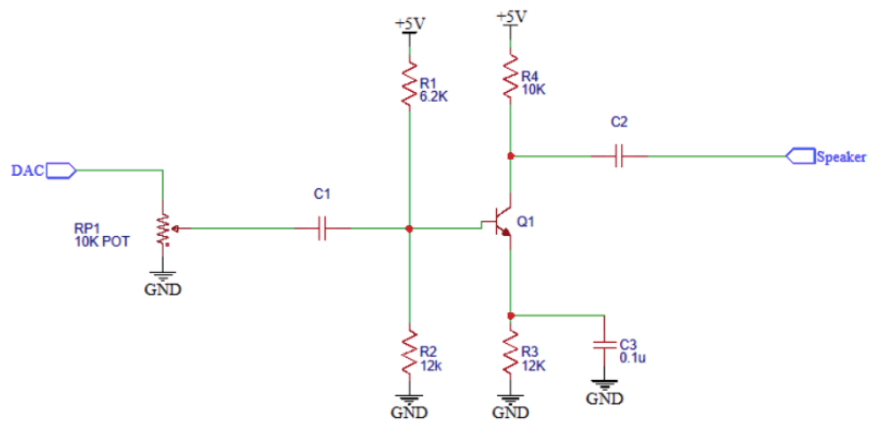


Figure 2: Amplifier Design 2

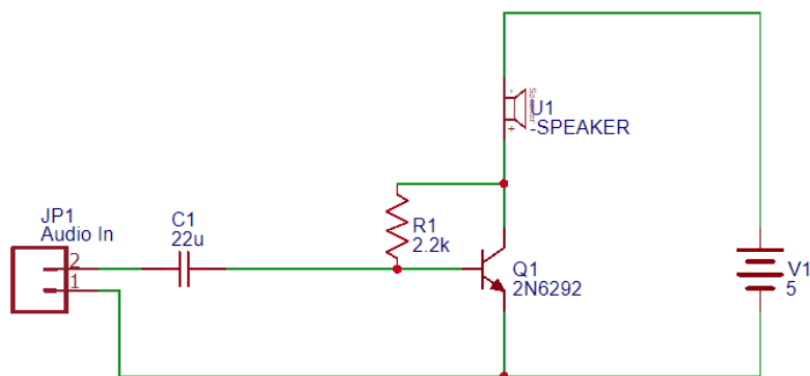


Figure 3: Amplifier Design 3

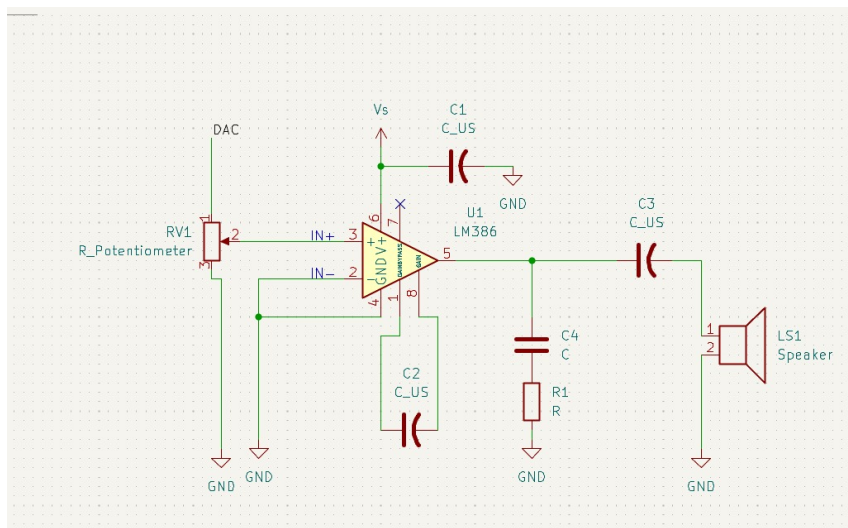


Figure 4: Amplifier Design 4

Task 7

Connect PB0 (TIM3_CH3) to the input of your amplifier. Make sure that the STM32 and your amplifier share a ground line. Test all three of your waveforms and three .wav files.

Task 8

This Task is purely optional. Attempt to redo the prac but this time making use of the inbuilt STM32 DAC instead.

A helpful resource to assist with this task can be found here: [DAC](#)

7 Submission

1. You will need to submit your main.c file in pdf format to Practical 4 Demonstration Assignment on Gradescope before your demo. It is mandatory at the top of your code to include the following (Check the Appendix 7 for example):

```
1      /*
2      1. Link: https://github.com/EEE3096S-UCT/EEE3096S-2025
3
4      2. Group Number: ##
5
6      3. Members: STDNUM001 STDNUM002
7      */
8      //Rest of your code
```

2. For your report once you are done submit the pdf to Practical 4 Report assignment on Gradescope. Remember to adhere to the naming convention.

If you want to replicate the code styling on Latex as the one in this prac manual do the following:

```
1 % Make sure to include the following packages in your document
2 \usepackage{xcolor}
3 \usepackage{minted}
4
5 Here is an example of C code:
6
7 \begin{minted}[frame=lines, linenos, bgcolor=black!5]{c}
8 /*
9 1. Link: https://github.com/EEE3096S-UCT/EEE3096S-2025
10 2. Group Number: ##
11 3. Members: STDNUM001 STDNUM002
12 */
13 // Rest of your code
14 \end{minted}
15
16 Rest of your report
```

That will display the C code as:

```
1      /*
2      1. Link: https://github.com/EEE3096S-UCT/EEE3096S-2025
3      2. Group Number: ##
4      3. Members: STDNUM001 STDNUM002
5      */
6      // Rest of your code
7      int main(){
8
9      }
```

Appendix: Report Structure

Section	Description
Introduction	Briefly introduce the aim and objectives of the practical and summarise your work briefly.
Methodology	Detail the steps undertaken and methods used to achieve the practical task in a logical and coherent manner.
Results and Discussion	Present the results and discuss their significance.
Conclusion	Summarise the work you did for the practical and any improvements you could make to your implementation.
AI Clause	In one paragraph discuss how you used LLMs while working on the practical and if you did find them useful in an embedded systems programming context.

Appendix: Code Submission Example

```
1  /*
2
3  1. Link: https://github.com/EEE3096S-UCT/EEE3096S-2025
4
5  2. Group Number: ##
6
7  3. Members: STDNUM001 STDNUM002
8
9  */
10
11
12 /* USER CODE BEGIN Header */
13 /**
14  *****
15  * @file           : main.c
16  * @brief          : Main program body
17  *****
18  * @attention
19  *
20  * Copyright (c) 2025 STMicroelectronics.
21  * All rights reserved.
22  *
23  * This software is licensed under terms that can be found in the LICENSE file
24  * in the root directory of this software component.
25  * If no LICENSE file comes with this software, it is provided AS-IS.
26  *
27  *****
28  */
29 /* USER CODE END Header */
30 /* Includes -----*/
31 #include "main.h"
32
33 /* Private includes -----*/
34 /* USER CODE BEGIN Includes */
35
36 /* USER CODE END Includes */
37
38 /* Private typedef -----*/
39 /* USER CODE BEGIN PTD */
40
41 /* USER CODE END PTD */
42
43 /* Private define -----*/
44 /* USER CODE BEGIN PD */
45
46 /* USER CODE END PD */
47
48 /* Private macro -----*/
49 /* USER CODE BEGIN PM */
50
51 /* USER CODE END PM */
52
53 /* Private variables -----*/
54
55 /* USER CODE BEGIN PV */
```

```

56
57  /* USER CODE END PV */
58
59  /* Private function prototypes -----*/
60  void SystemClock_Config(void);
61  static void MX_GPIO_Init(void);
62  /* USER CODE BEGIN PFP */
63
64  /* USER CODE END PFP */
65
66  /* Private user code -----*/
67  /* USER CODE BEGIN 0 */
68
69  /* USER CODE END 0 */
70
71  /**
72   * @brief The application entry point.
73   * @retval int
74   */
75  int main(void)
76  {
77      /* USER CODE BEGIN 1 */
78
79      /* USER CODE END 1 */
80
81      /* MCU Configuration-----*/
82
83      /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
84      HAL_Init();
85
86      /* USER CODE BEGIN Init */
87
88      /* USER CODE END Init */
89
90      /* Configure the system clock */
91      SystemClock_Config();
92
93      /* USER CODE BEGIN SysInit */
94
95      /* USER CODE END SysInit */
96
97      /* Initialize all configured peripherals */
98      MX_GPIO_Init();
99      /* USER CODE BEGIN 2 */
100
101      /* USER CODE END 2 */
102
103      /* Infinite loop */
104      /* USER CODE BEGIN WHILE */
105      while (1)
106      {
107
108          /* USER CODE END WHILE */
109
110          /* USER CODE BEGIN 3 */
111          // Toggle the LEDs with a 1s delay
112          // Code to check if the board is working
113          HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3

```

```

114 |GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7);
115
116         HAL_Delay(1000);
117     }
118     /* USER CODE END 3 */
119 }
120
121 /**
122  * @brief System Clock Configuration
123  * @retval None
124  */
125 void SystemClock_Config(void)
126 {
127     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
128     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
129
130     /** Configure the main internal regulator output voltage
131      */
132     __HAL_RCC_PWR_CLK_ENABLE();
133     __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);
134
135     /** Initializes the RCC Oscillators according to the specified parameters
136      * in the RCC_OscInitTypeDef structure.
137      */
138     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
139     RCC_OscInitStruct.HSISState = RCC_HSI_ON;
140     RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
141     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
142     if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
143     {
144         Error_Handler();
145     }
146
147     /** Initializes the CPU, AHB and APB buses clocks
148      */
149     RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
150         |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
151     RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
152     RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
153     RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
154     RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
155
156     if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
157     {
158         Error_Handler();
159     }
160 }
161
162 /**
163  * @brief GPIO Initialization Function
164  * @param None
165  * @retval None
166  */
167 static void MX_GPIO_Init(void)
168 {
169     GPIO_InitTypeDef GPIO_InitStruct = {0};
170     /* USER CODE BEGIN MX_GPIO_Init_1 */
171     /* USER CODE END MX_GPIO_Init_1 */

```

```

172
173  /* GPIO Ports Clock Enable */
174  __HAL_RCC_GPIOB_CLK_ENABLE();
175
176  /*Configure GPIO pin Output Level */
177  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
178                      |GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7, GPIO_PIN_RESET);
179
180  /*Configure GPIO pins : PB0 PB1 PB2 PB3
181                          PB4 PB5 PB6 PB7 */
182  GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
183                      |GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7;
184  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
185  GPIO_InitStruct.Pull = GPIO_NOPULL;
186  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
187  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
188
189  /* USER CODE BEGIN MX_GPIO_Init_2 */
190  /* USER CODE END MX_GPIO_Init_2 */
191  }
192
193  /* USER CODE BEGIN 4 */
194
195  /* USER CODE END 4 */
196
197  /**
198   * @brief This function is executed in case of error occurrence.
199   * @retval None
200   */
201  void Error_Handler(void)
202  {
203      /* USER CODE BEGIN Error_Handler_Debug */
204      /* User can add his own implementation to report the HAL error return state */
205      __disable_irq();
206      while (1)
207      {
208      }
209      /* USER CODE END Error_Handler_Debug */
210  }
211
212  #ifdef USE_FULL_ASSERT
213  /**
214   * @brief Reports the name of the source file and the source line number
215   *        where the assert_param error has occurred.
216   * @param file: pointer to the source file name
217   * @param line: assert_param error line source number
218   * @retval None
219   */
220  void assert_failed(uint8_t *file, uint32_t line)
221  {
222      /* USER CODE BEGIN 6 */
223      /* User can add his own implementation to report the file name and line number,
224       ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
225      /* USER CODE END 6 */
226  }
227  #endif /* USE_FULL_ASSERT */

```